

SPM: Message Passing Interface (MPI)

(Part III)

Academic year 2025-2026

Prof. Massimo Torquati
massimo.torquati@unipi.it

Outline

- Some advanced MPI features:
 - Groups and Communicators
 - MPI derived data types
- MPI and threads

Advanced MPI features solve two main problems:

1. defining who communicates with whom, via groups, communicators, and topologies
2. defining what memory layout is communicated, via derived datatypes

Groups

- A group is an ordered set of processes
- Each process in a group is associated with a **unique** rank (from 0 to groupsize - 1)
- A group is typically associated with a communicator object, **which defines the context**
 - The group of a communicator can be obtained by calling: `int MPI_Comm_group(MPI_Comm comm, MPI_Group *group);`
- MPI provides operations to construct new process groups based on existing groups
 - `MPI_Group_union()`, `MPI_Group_intersection()`, `MPI_Group_difference()`, `MPI_Group_incl()`, `MPI_Group_excl()`
- The code snippet splits the application group with p processes in two groups (even and odd)

```
int even = (p+1)/2;
for(i=0; i < even; ++i) members[i] = 2 * i;
MPI_Group world_group, even_group, odd_group;
MPI_Comm_group(MPI_COMM_WORLD, &world_group);
MPI_Group_incl(world_group, even, members, &even_group);
MPI_Group_excl(world_group, even, members, &odd_group);
```


Communicators

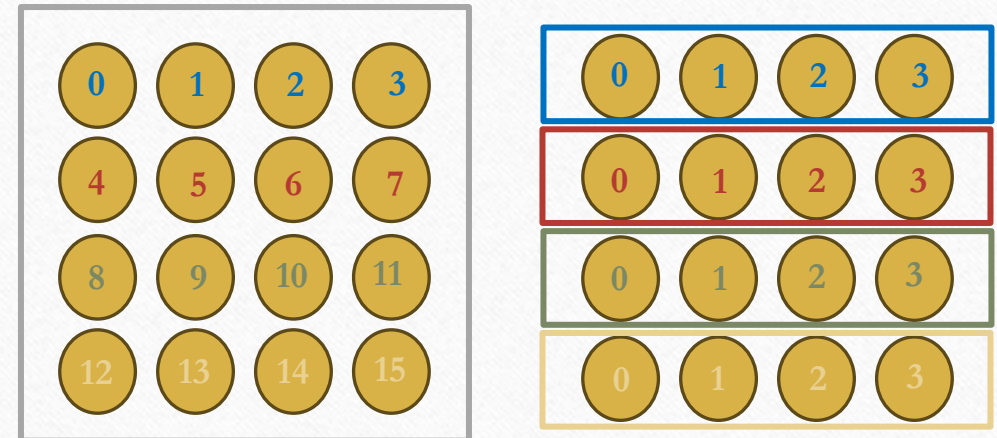
- A communicator encompasses a group of processes that can communicate
- A communicator binds a **process group** and a communication **contexts**
 - The communication context prevents messages in different communicators from matching
 - **Intracommunicator**: used for communication within a group
 - **Intercommunicator**: used for communications between groups
- Purposes: collective communications, **user-defined virtual topologies**
- MPI provides more than 40 functions/routines related to groups, communicators and virtual topologies. Some of them related to communicator management are:
 - `int MPI_Comm_create(MPI_Comm old_comm, MPI_Group group, MPI_Comm* new_comm)`
 - `int MPI_Comm_compare(MPI_Comm comm1, MPI_Comm comm2, int* res)`
 - `int MPI_Comm_dup(MPI_Comm comm, MPI_Comm* new_comm)`
 - `int MPI_Comm_split(MPI_Comm comm, int color, int key, MPI_Comm* new_comm)`

Splitting Communicators

- `int MPI_Comm_split(MPI_Comm comm, int color, int key, MPI_Comm* new_comm)`
 - Useful to partition communication among process subsets. **It is a collective operation!**
 - It partitions `comm` into disjoint subgroups, **one for each value of color**
 - Within each group, the MPI processes are ranked in the order defined by the value of the **key**
 - If all keys are the same, then all processes will have the relative rank order as they have in the parent group

```
int rank;  
MPI_Comm_rank(MPI_COMM_WORLD, &rank);  
int color = rank/4;      // one color for each logical row  
int key  = rank%4;      // rank order inside the row communicator  
MPI_Comm row_comm;      // new row communicator  
MPI_Comm_split(MPI_COMM_WORLD, color, rank, &row_comm);
```

Processes with the same color enter the same new communicator. The key defines the rank order inside the new communicator, not the new rank.



Virtual Topologies (brief mention)

- A virtual topology represents the way that MPI processes communicate
 - logical process arrangement in regular topological patterns such as 2D or 3D grid
 - logical process arrangement can also be described by a graph
- **Cartesian topologies:** for regular problems, MPI provides a convenient multidimensional mesh organization
 - `int MPI_Cart_create(MPI_Comm comm_old, int ndims, int* dims, int* periods, int reorder, MPI_Comm* comm_cart)`
- **Graph Topologies:**
 - For irregular graphs, MPI also provides distributed graph topologies
 - `int MPI_Graph_create(MPI_Comm comm_old, int nnodes, int* index, int* edges, int reorder, MPI_Comm* cgraph)`
- More info: Section 8 of the MPI 4.1 standard specification

MPI Derived Data Types

- MPI allows users to build new, **user-defined datatypes based on primitive MPI datatypes**
 - A derived datatype describes the layout. It does not copy the data by itself
 - The actual data movement happens when the datatype is used in a communication call
- Derived datatypes are useful in situations where:
 - the data to send are non-contiguous in memory (e.g., columns of a row-major matrix)
 - the data is contiguous in memory but made of mixed types (e.g., a struct containing an integer and an array of floats)
 - Such data can be sent using MPI_BYTE. However, using explicit datatypes improves program readability **and often avoids extra packing code**
- They may enhance performance by reducing the number of memory copies and the number of messages
- A **general datatype** is an opaque object that specifies two things:
 - A sequence of basic datatypes
 - A sequence of displacements defining where each block is located
- Order of items does not need to coincide with their order in memory. An item may appear more than once

Building a datatype

- The primary steps are:
 1. Construct the datatype using a constructor, for example `MPI_Type_vector()`, The new datatype has type `MPI_Datatype`
 2. Commit the datatype using `MPI_Type_commit()`, so it can be used in communication calls
 3. Once the datatype is not used anymore, free it `MPI_Type_free()`
- Construction and commit are mandatory before using the datatype, the release of the datatype is optional but recommended

```
MPI_Datatype new_type;           // declare the new_type datatype
...
MPI_Type_vector(..., &new_type); // construct new_type
MPI_Type_commit(&new_type);      // commit new_type

...

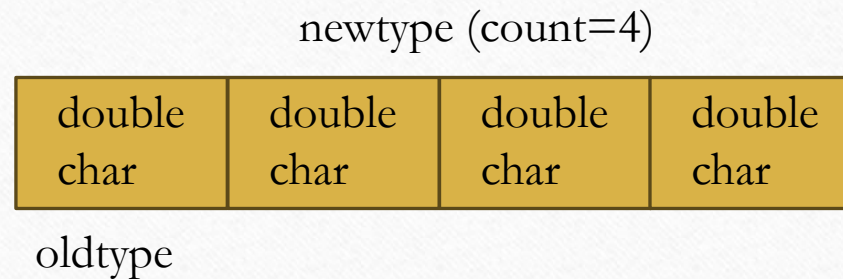
MPI_Type_free(&new_type);        // free new_type
```


Datatype constructors

- The most commonly used datatypes constructors are:
 - `int MPI_Type_contiguous(int count, MPI_Datatype oldtype, MPI_Datatype* newtype)`
 - `int MPI_Type_vector(int count, int blocklen, int stride, MPI_Datatype oldtype, MPI_Datatype* newtype)`
 - `int MPI_Type_create_struct(int count, int blocklen[], MPI_Aint displacements[], MPI_Datatype oldtype[], MPI_Datatype* newtype)`
 - `MPI_Aint` is an MPI integer type large enough to store memory addresses and byte displacements
 - `int MPI_Type_indexed(int count, int blocklen[], int displacements[], MPI_Datatype oldtype, MPI_Datatype* newtype)`
- Displacement units depend on the constructor, for example, in `MPI_Type_indexed`, displacements are expressed in extents of oldtype, not bytes. For byte displacements, use `MPI_Type_create_hindexed`

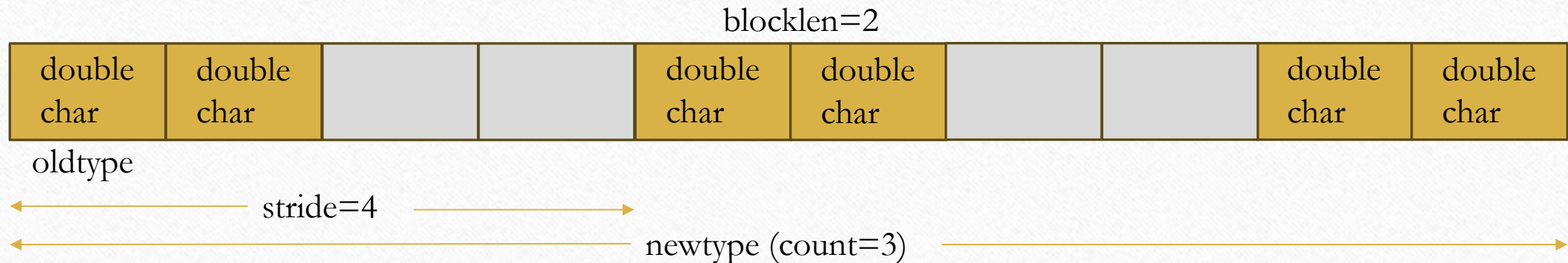
Contiguous datatype

- `int MPI_Type_contiguous(int count, MPI_Datatype oldtype, MPI_Datatype* newtype)`
- The `newtype` is the datatype obtained by concatenating `count` copies of `oldtype`
- Example:



Vector datatype

- `int MPI_Type_vector(int count, int blocklen, int stride, MPI_Datatype oldtype, MPI_Datatype* newtype)`
- Create a datatype from elements of an existing data type
 - `count` is the number of blocks
 - `blocklen` is the number of elements in each block
 - `stride` is the number of elements between the start of each block. It is measured in extents of `oldtype`, not bytes
 - To use bytes displacements, use `MPI_Type_create_hvector()`



Structure datatype

- `int MPI_Type_create_struct(int count, int blocklen[], MPI_Aint displacements[], MPI_Datatype oldtypes[], MPI_Datatype* newtype)`
- Create a datatype from a set of datatypes. Each block is a collection of data of the same type
 - `count` is the number of blocks
 - `blocklen` is an array of integers setting the size of each block
 - `displacements` is an array setting the relative offset for each block (**expressed in bytes!**)
 - Set displacements manually is unsafe and not portable, use `MPI_Get_address()`

```
struct Particle {  
    float x;  
    float y;  
    int type;  
} p;
```

```
int blocklen[] = {2, 1}; // 2,1 -> two float, one int  
MPI_Datatype oldtypes[] = {MPI_FLOAT, MPI_INT};  
MPI_Aint xaddr, typeaddr, displs[2];  
MPI_Get_address(&p.x, &xaddr);  
MPI_Get_address(&p.type, &typeaddr);  
displs[0] = 0;  
displs[1] = typeaddr - xaddr;  
MPI_Type_create_struct(2, blocklen, displs, oldtypes, &new_type);  
MPI_Type_commit(&new_type);
```

If the datatype is used with `count > 1` to send an array of structs, check the extent. Often `MPI_Type_create_resized` is needed to make the extent equal to `sizeof(struct)`.

Application examples

- The following examples show where communicators and derived datatypes become useful in real codes
 - `mpi_jacobi_1d_blk.cpp` Jacobi iteration algorithm using blocking send/recv
 - `mpi_jacobi_1d_nblk.cpp` Jacobi iteration algorithm using non-blocking send/recv
 - `mpi_mm2d.cpp` Simple 2D-based Matrix Multiplication
 - `mpi_summa.cpp` Scalable Matrix Multiplication

Example: Jacobi iterations

Initial data

-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1
3	3	3	3	3	3	3	3	3	3	3	3
3	3	3	3	3	3	3	3	3	3	3	3
2	2	2	2	2	2	2	2	2	2	2	2
2	2	2	2	2	2	2	2	2	2	2	2
2	2	2	2	2	2	2	2	2	2	2	2
1	1	1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1	1	1	1
0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0
-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1

stencil type
(5-point stencil)



Proc 0

-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1
3	3	3	3	3	3	3	3	3	3	3	3
3	3	3	3	3	3	3	3	3	3	3	3
2	2	2	2	2	2	2	2	2	2	2	2

Proc 1

2	2	2	2	2	2	2	2	2	2	2	2
2	2	2	2	2	2	2	2	2	2	2	2
1	1	1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1	1	1	1

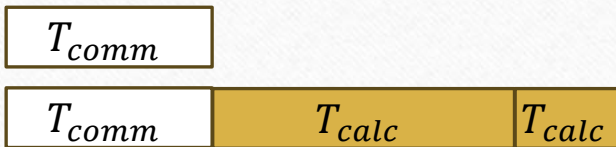
Proc 2

1	1	1	1	1	1	1	1	1	1	1	1
0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0
-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1

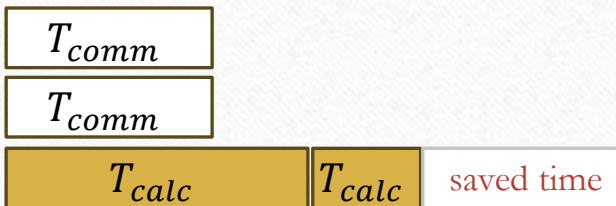
- Neighboring processes exchange **halo** rows needed to update boundary points. **Proc1** needs the last row from **Proc0** and the first row from **Proc2** to compute the stencil on the elements of the first and last rows of its partition. **Proc0** needs the first row from **Proc1**. **Proc2** needs the last row of **Proc1**.

Overlapping computation & communication

no overlap (best case with blocking communications)



overlap (best case with non-blocking communications)



$$T_C = T_{calc} + T_{comm} \quad \text{no overlap}$$
$$T_C = \max(T_{calc}, T_{comm}) \quad \text{overlap}$$

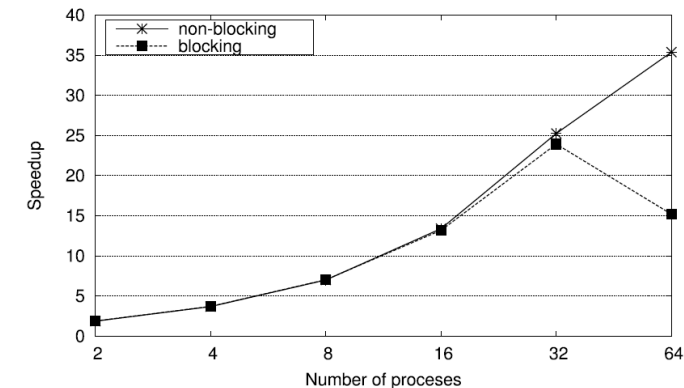


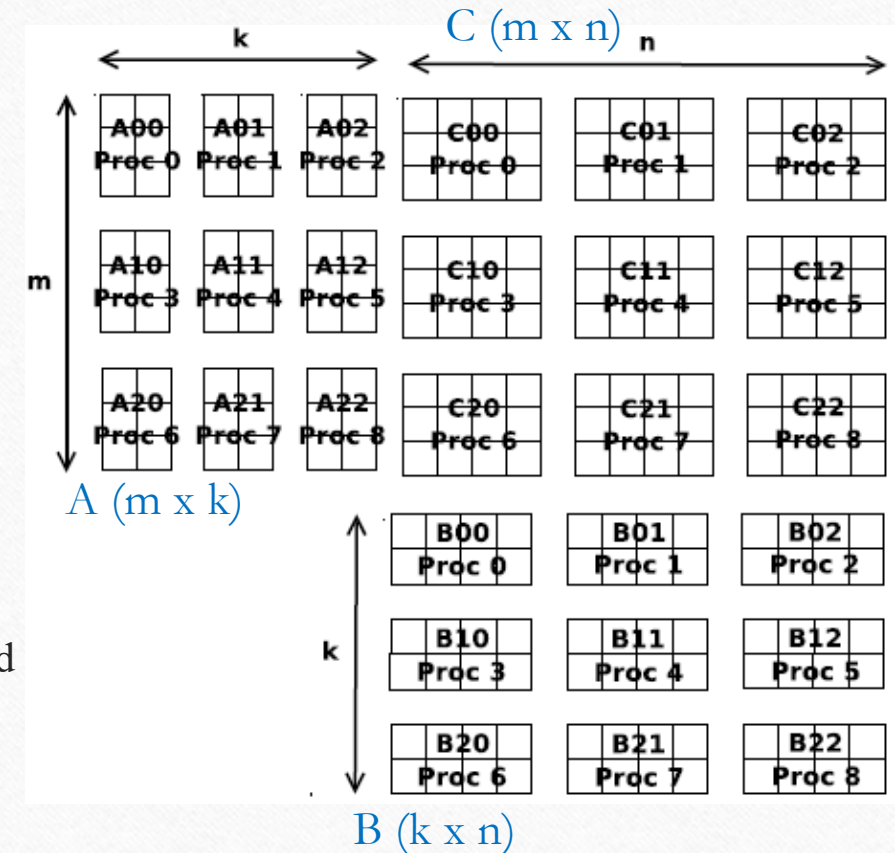
FIGURE 9.11

Performance of the [Jacobi](#) parallelization with blocking and nonblocking communications on a system with 64 cores. Matrix dimensions are 4096×4096 and error threshold is 0.1.

MPI non-blocking communication enables overlap, **but does not guarantee it**. Actual overlap depends on MPI progress, message size, and available independent computation

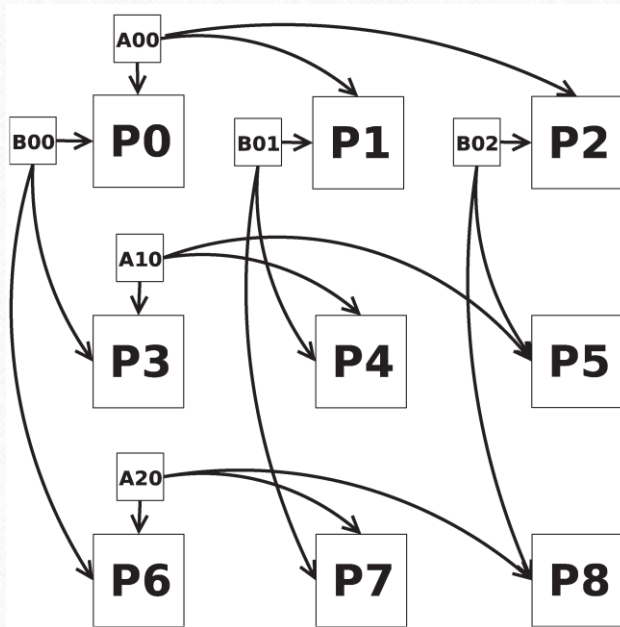
Complex Communicators: SUMMA

- Scalable Universal Matrix Multiplication (SUMMA)
- The matrices A, B, and C are initially distributed among processes
- Not all data to compute the partial product are stored in the local memory of processes
 - $C_{01} = A_{00} \times B_{01} + A_{01} \times B_{11} + A_{02} \times B_{21}$
- Rows and Columns group communicators are used to obtain all data needed locally (through broadcast phases)
 - During the algorithm, panels of A and B are temporarily replicated along process rows/columns through broadcasts

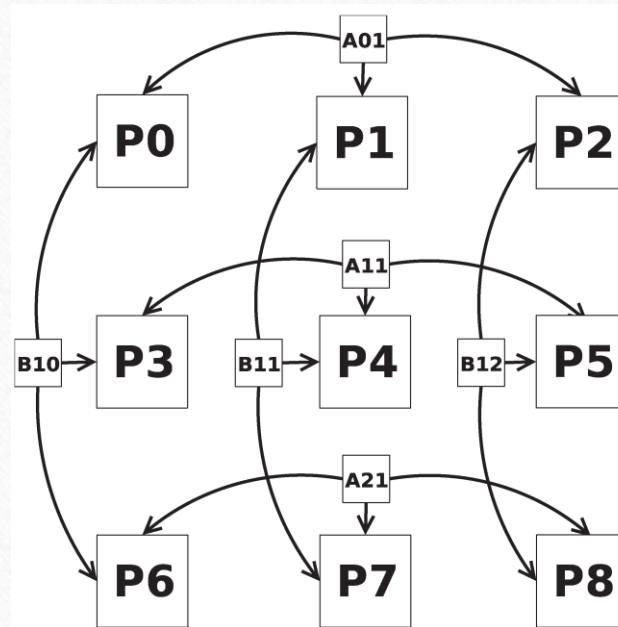


Complex Communicators: SUMMA

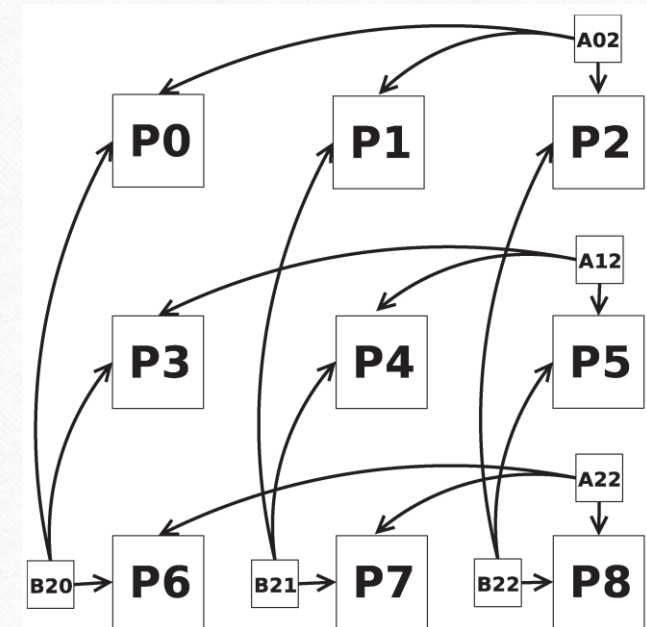
- Processes organized in a 3×3 grid: 3 broadcast phases (in general k phases for a grid $k \times k$)



phase1



phase2



phase3

- Horizontal broadcast for A, vertical broadcast for B

Complex Communicators: SUMMA

- Splitting `MPI_COMM_WORLD`

```
// Create the communicators by splitting MPI_COMM_WORLD
MPI_Comm rowComm;
MPI_Comm colComm;
//
//          color          key
MPI_Comm_split(MPI_COMM_WORLD, myId/gridDim, myId%gridDim, &rowComm);
MPI_Comm_split(MPI_COMM_WORLD, myId%gridDim, myId/gridDim, &colComm);
```

Table 9.1 color and key for each process to create the communicators for rows and columns over a 3×3 grid.

Process ↓	Row Communicator		Column Communicator	
	Color	Key	Color	Key
0	0	0	0	0
1	0	1	1	0
2	0	2	2	0
3	1	0	0	1
4	1	1	1	1
5	1	2	2	1
6	2	0	0	2
7	2	1	1	2
8	2	2	2	2

The same row/column communicators could also be derived from a cartesian communicator

Threads and MPI

- MPI describes parallelism between processes with separate address spaces
- `MPI_Init` provides only the basic single-thread initialization
 - OpenMPI needs to be compiled with `--enable-mpi-threads`
 - If there is no thread support, **only one thread at a time may execute MPI routines**
- **Threads in MPI are not addressable**
 - It is **not** possible to send a message to a specific thread of a process
- To init MPI to support threads **`MPI_Init_thread` must be used** instead of `MPI_Init`
- The MPI process can use threads for computation and/or communication
 - e.g., MPI+OpenMP, MPI+C++ threads, MPI+FastFlow, MPI+CUDA, etc.
 - The paradigm **MPI + “X”** is known as “**hybrid programming**”
 - NOTE: X could also be **MPI Shared Memory** (introduced in MPI-3, for true on-node shared-memory regions)

Initialize MPI for threading

- `int MPI_Init_thread(int* argc, char*** argv, int required, int* provided)`
- The argument `required` is used to request the desired level of thread support. It can be:
 - `MPI_THREAD_SINGLE`, the process is single-threaded. No other thread will call MPI routines
 - `MPI_Init` is equivalent to `MPI_Init_thread(..., MPI_THREAD_SINGLE, ...)`
 - `MPI_THREAD_FUNNELED`, only the thread that called `MPI_Init_thread` may make MPI calls, e.g., if OpenMP is used, only the Master thread executes MPI function calls.
 - `MPI_THREAD_SERIALIZED`, Multiple threads may call MPI, but never concurrently. The application must serialize the calls.
 - `MPI_THREAD_MULTIPLE`, multiple threads may call MPI functions with no restrictions

The MPI runtime fills the `provided` argument with the actual thread support offered by the implementation.

NOTE: could be less than `required`!

```
int provided=0;
MPI_Init_thread(&argc, &argv, MPI_THREAD_FUNNELED, &provided);

if (provided < MPI_THREAD_FUNNELED)
    MPI_Abort(MPI_COMM_WORLD, 1);
```


Threaded MPI

- Section 11.6 of the MPI 4.1 standard specification
- When multiple threads make MPI calls concurrently, the outcome will be as if the calls are executed **sequentially in some order**.
 - User must ensure that collective operations are correctly ordered among threads (do not let different threads issue collectives on the same communicator unless you can guarantee the same ordering everywhere)
 - User must ensure data race free MPI programs

The two main requirements for a thread-compliant implementation are listed below.

1. All MPI calls are **thread-safe**, i.e., two concurrently running threads may make MPI calls and the outcome will be as if the calls executed in some order, even if their execution is interleaved.
2. Blocking MPI calls will block the calling thread only, allowing another thread to execute, if available. The calling thread will be blocked until the event on which it is waiting occurs. Once the blocked communication is enabled and can proceed, then the call will complete and the thread will be marked runnable, within a finite time. A blocked thread will not prevent *progress* of other runnable threads on the same process, and will not prevent them from executing MPI calls.

Example 11.19. Process 0 consists of two threads. The first thread executes a blocking send call `MPI_Send(buff1, count, type, 0, 0, comm)`, whereas the second thread executes a blocking receive call `MPI_Recv(buff2, count, type, 0, 0, comm, &status)`, i.e., the first thread sends a message that is received by the second thread. This communication should always succeed. According to the first requirement, the execution will correspond to some interleaving of the two calls. According to the second requirement, a call can only block the calling thread and cannot prevent progress of the other thread. If the send call went ahead of the receive call, then the sending thread may block, but this will not prevent the receiving thread from executing. Thus, the receive call will occur. Once both calls occur, the communication is enabled and both calls will complete. On the other hand, a single-threaded process that posts a send, followed by a matching receive, may deadlock. The progress requirement for multithreaded implementations is stronger, as a blocked call cannot prevent progress in other threads.

Threaded MPI

	P0	P1
Thread 0	<code>MPI_Recv(... 1...);</code>	<code>MPI_Recv(... 0 ...);</code>
Thread 1	<code>MPI_Send(... 1...);</code>	<code>MPI_Send(... 0 ...);</code>

- With `MPI_THREAD_MULTIPLE`, the MPI library must allow concurrent calls from multiple threads and guarantee that one thread's blocking call cannot starve progress in another thread. In the example, no matter which thread runs first, the two sends and two receives will always match and complete
- This does not come for free. The implementation must protect certain code and data structures with mutexes and this usually introduces some overhead
 - `MPI_THREAD_MULTIPLE` makes concurrent MPI calls legal, not necessarily fast

Example: MPI+OpenMP

- Pay attention to process-node bindings and OpenMP threads affinity

- Example:

```
> salloc -N 8
```

```
> mpirun -x OMP_NUM_THREADS=16 -x OMP_DISPLAY_AFFINITY=true --map-by node --bind-to=none  
--report-bindings -n 8 ./mpi_trapezoid+omp 200000000
```

round-robin distribution
of processes to nodes

socket/core/none

NOTE: env variables can be passed to the application when launching it with **srun** through the **--export** command line option. For example:

```
srun --mpi=pmix --export=ALL,OMP_NUM_THREADS=16 --cpu-bind=none ./mpi_trapezoid+omp 2000000000
```

Suggested Readings

- Chapter 9 of the «Parallel Programming Concepts and Practice» course book
- MPI: A Message-Passing Interface Standard Version 4.1
 - <https://www.mpi-forum.org/docs/mpi-4.1/mpi41-report.pdf>
- OpenMPI documentation:
 - <https://docs.open-mpi.org/en/v5.0.x/>
- MPI tutorial
 - <https://hpc-tutorials.llnl.gov/mpi/>
- OSU micro-benchmarks:
 - <https://mvapich.cse.ohio-state.edu/benchmarks/>
 - C benchmarks installed in /opt/ohpc/pub/mpi/osu-7.4/ on the spmcluster